



SHERPA: A Model-Driven Framework for Large Language Model Execution

Boqi Chen^{*}, Kua Chen^{*}, José Antonio Hernández López[†]
Gunter Mussbacher[‡], Dániel Varró^{‡§}, Amir Feizpour[¶]

^{*}Electrical and Computer Engineering, McGill University, Canada - {firstname.lastname}@mail.mcgill.ca

[†]Department of Computer Science and Systems, University of Murcia, Spain - joseantonio.hernandez6@um.es

[‡]Electrical and Computer Engineering, McGill University, Canada - {firstname.lastname}@mcgill.ca

[§]Department of Computer and Information Science, Linköping University, Sweden - daniel.varro@liu.se

[¶]Aggregate Intellect Inc., Canada - amir@ai.science

Abstract—Recently, large language models (LLMs) have achieved widespread application across various fields. Despite their impressive capabilities, LLMs suffer from a lack of structured reasoning ability, particularly for complex tasks requiring domain-specific best practices, which are often unavailable in the training data. Although multi-step prompting methods incorporating human best practices, such as chain-of-thought and tree-of-thought, have gained popularity, they lack a general mechanism to control LLM behavior. In this paper, we propose *SHERPA*, a model-driven framework to improve the LLM performance on complex tasks by explicitly incorporating domain-specific best practices into hierarchical state machines. By structuring the LLM execution processes using state machines, *SHERPA* enables more fine-grained control over their behavior via rules or decisions driven by machine learning-based approaches, including LLMs. We show that *SHERPA* is applicable to a wide variety of tasks—specifically, code generation, class name generation, and question answering—replicating previously proposed approaches while further improving the performance. We demonstrate the effectiveness of *SHERPA* for the aforementioned tasks using various LLMs. Our systematic evaluation compares different state machine configurations against baseline approaches without state machines. Results show that integrating well-designed state machines significantly improves the quality of LLM outputs, and is particularly beneficial for complex tasks with well-established human best practices but lacking data used for training LLMs.

Index Terms—state machine, large language model, structured reasoning, best practice integration.

I. INTRODUCTION

Context. Models capturing structural system behavior are fundamental elements of model-driven engineering (MDE). Behavioral models such as hierarchical state machines [1], activity diagrams [2], and sequence diagrams [2] provide well-defined structures to represent system behavior at various levels of abstraction. Within the context of MDE, these models facilitate the mapping of requirements to system design by leveraging human best practices and domain knowledge. Subsequently, these models enable the generation of lower-level artifacts. They also support the verification of system behavior, which is essential for safety-critical applications.

Partially supported by the FRQNT-B2X project (file number: 319955), IT30340 Mitacs Accelerate, and the Wallenberg AI, Autonomous Systems and Software Program (WASP), Sweden

Large language models (LLMs) produce stochastic sequences of predictions based on input prompts [3]. With the emergence of advanced LLMs such as GPT-4o [4] and Qwen-2.5 [5], they have become powerful tools for automating complex tasks across diverse domains using natural language. LLMs have shown remarkable capabilities in various applications, including code generation [6], question answering [7], and planning [8], [9]. In MDE, recent efforts have also shown promising results in fully automating the generation of diverse types of models directly from natural language descriptions [10].

Problem description. However, the stochastic nature of LLMs raises significant concerns around the potential hallucination in the generated outputs [11]. Moreover, specific human best practice is rarely reflected in training data, which limits LLMs' performance in complex and domain-specific tasks. Approaches like retrieval augmented generation (RAG) provide a way to add up-to-date task context [12]–[15], but they do not offer an approach to solve tasks involving complex workflows. Despite recent advancements in multi-step reasoning approaches such as Chain-of-Thought [16], Tree-of-Thought [17], and ReAct [18], LLMs still face challenges in maintaining long-term consistency and performing effective multi-step planning for complex tasks.

Recently, integrating structured workflows into the execution of LLM-based applications has been recognized as a promising solution to address these limitations. Multiple frameworks [19], [20] have been proposed to integrate LLMs with behavioral models to enhance task performance. However, the model used in these approaches still remains simple and is often coupled with implementation. More expressive behavior models, such as hierarchical state machines, which provide flexible and powerful mechanisms to decompose tasks into simpler sub-tasks, have not yet been fully leveraged. Moreover, best practices can often be modeled in multiple ways, which may significantly influence the performance of LLM systems, making it important for the framework to enable rapid experimentation.

Objectives. This paper investigates the impact of model-driven processes on LLM task execution using behavioral models to represent human best practices. We explore the potential of these models to enhance the performance of LLMs for complex tasks, by proposing *SHERPA*: a framework for

Stateful Hierarchical Execution and Reasoning for Process Automation. SHERPA leverages hierarchical state machines to explicitly incorporate best practices into the task execution process. LLMs can be used within actions performed during state transitions. Additionally, SHERPA supports flexible task execution by enabling state transitions driven by logic and stochastic predictions, while explicitly preserving execution histories and intermediate results for subsequent executions.

Contribution. This paper makes the following contributions:

- We propose SHERPA, a model-driven framework that integrates human best practices as hierarchical state machines with LLMs to enhance task performance.
- We propose a hybrid policy for state transition that combines rule-based and LLM-driven approaches while proposing a belief structure to store the execution history and intermediate results.
- We show the applicability of SHERPA to a variety of tasks, including code generation, class name generation, and question answering. We demonstrate how SHERPA can be used to replicate existing approaches while improving their performance by enhancing the state machine design.
- We evaluate the effectiveness of SHERPA on these three tasks using two LLM families: GPT-4o [4] and Qwen-2.5 [5] as well as task-specific LLMs such as Qwen-2.5-Coder [21] and Llama3.1-70B [22].

Added value. Compared to existing approaches combining LLMs with structured workflows, SHERPA is a more general, model-driven framework for tackling complex tasks with LLMs. It showcases how current LLMs can benefit from MDE approaches by enabling task decomposition with modular representation. SHERPA decouples the state machine design from the action implementation, allowing changing state machine designs without updating the implementation.

II. BACKGROUND

LLMs. Large language models (LLMs) have gained significant attention for many tasks based on natural language [6], [7], [10]. LLMs use deep neural networks, typically with transformer architecture [23], to estimate a probability distribution of text sequences. Current state-of-the-art generative LLMs are primarily based on the decoder-only architecture, which is derived from the original transformer architecture. In general, an LLM llm can be seen as a mapping between sequences of input and output tokens: $\text{llm}: s_{in} \rightarrow s_{out}$. More specifically, given a sequence of tokens (called *prompt*) $s_{in} = \{s_1, s_2, \dots, s_{k-1}\}$, LLMs estimate the conditional probability of the next token $P(s_k | s_1, \dots, s_{k-1})$. Then LLMs continue to generate the next tokens auto-regressively until a special *stop* token is generated or a maximum length is reached to output the final sequence s_{out} . These LLMs are typically first pre-trained on a large corpus of text data using unsupervised learning, followed by fine-tuning to follow input instructions [24], [25].

LLM frameworks. Recent studies indicate that augmenting LLMs with external tools can enhance performance across various tasks [26]. Such integration aims to separate task

planning and decomposition with LLMs from task solving with other tools [18] to address specific problems more effectively.

However, the performance of LLMs may be limited in tasks requiring domain-specific knowledge not already available in their training data. For instance, within the context of software modeling, LLMs struggle to generate domain models that adhere to established best practices and modeling patterns [27]. To overcome this limitation, recent LLM frameworks support the construction of structured workflows to guide the behavior of LLMs [19], [28]. In these frameworks, workflows are represented as graphs, with nodes corresponding to different tasks which can be potentially executed by an LLM, and edges denoting task progression. LLMs can also assist in making branching decisions at nodes within these workflows.

Furthermore, LLMs can be equipped with external memory to store and retrieve contextual information as needed [28], [29]. This capability addresses their limitations in handling tasks across multiple steps. Such memory implementations often utilize key-value stores [29], enabling the retrieval of relevant information at various task stages.

In this paper, we aim to combine these aspects for LLMs by leveraging a state-machine-driven approach. Each instance of SHERPA incorporates a hierarchical state machine to structure workflows, where actions within transitions may use external tools. During state machine execution, a *belief* is maintained as memory to retain and access pertinent information.

State machines. A state machine (SM) [30] is a model used to describe the behavior of a system that can be in a finite number of states. SMs provide a visual representation to help with the development and maintenance of a system. Moreover, they can be used to enable automated code generation [31], [32] and facilitate verification of system behaviors [33].

Formally, an SM is a tuple $(\mathcal{S}, \mathcal{T}, \mathcal{A}, \mathcal{G}, \text{entry}, \text{exit}, \delta, s_0, \mathcal{F})$ where $\mathcal{S}$ is a set of states, $\mathcal{T}$ is a set of event triggers for transitions, $\mathcal{A}$ is the set of actions that can be performed during transitions or in states, $\mathcal{G}$ is the set of guard conditions for transitions, $\text{entry}: \mathcal{S} \rightarrow \mathcal{A}$ and $\text{exit}: \mathcal{S} \rightarrow \mathcal{A}$ map states to entry and exit actions, respectively. $\delta: \mathcal{S} \times \mathcal{T} \times \mathcal{G} \rightarrow \mathcal{A} \times \mathcal{S}$ is the state transition function, $s_0 \in \mathcal{S}$ is the initial state, and $\mathcal{F} \subseteq \mathcal{S}$ is the set of final states.

A hierarchical SM is an extension of SMs [1] by organizing states hierarchically. When a state is activated, all its parent states are also considered active, and transitions defined for a parent state apply to its child states as well. This hierarchy improves modularity and abstraction, reducing redundancy in defining transitions between related states. Formally, a hierarchical SM adds a hierarchical function $h: \mathcal{S} \rightarrow \mathcal{S}$ that maps states to their parent states. The state transition function δ is extended to include the parent state in the transition tuple.

There exist many definitions of the textual languages for SMs, such as PlantUML [34] and Umple [35]. While SHERPA uses JSON to define the SM, it can be easily adapted to other concrete syntax by using a parser.

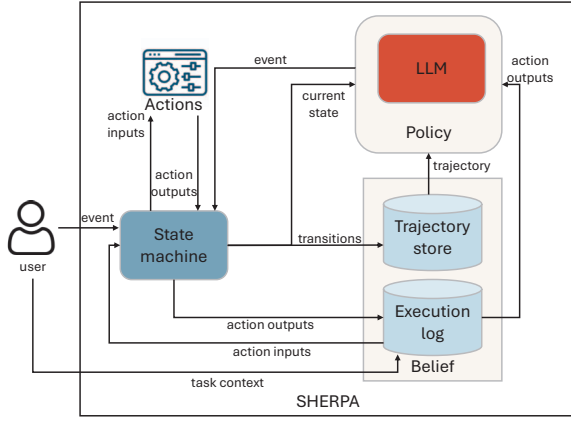


Fig. 1. Architecture for SHERPA

III. COMBINING STATE MACHINES WITH LLMs

This section introduces how SHERPA integrates SMs with LLM execution. We first provide an overview of the SHERPA framework, followed by a detailed description of the architecture with an example in question answering.

A. Overview

Figure 1 provides an overview of the architecture. We define each instance of SHERPA as an *agent* that is capable of executing a specific task. Each agent is associated with a *state machine*. A user, either a human or another agent, interacts with the agent by sending an *event* to its SM. These events can include messages, such as a question, which are recorded as *task context* in the agent's *belief*. Then the SM performs state transitions based on the received event and may execute predefined *actions* associated with that transition. Following the transition, the SM updates the agent's *belief* with the result of the action and the agent's new state. The belief contains the *trajectory store* keeping track of the taken *transitions* in the SM, and an *execution log* containing (1) executed actions including their *outputs* and (2) a key-value store containing information relevant to the subsequent states as *action inputs*.

This updated belief, along with available transitions in the *current state*, is then provided to the *policy*, which uses an LLM or a predefined rule to determine the optimal next step. The selected event is subsequently sent back to the SM to initiate further transitions. This cycle continues until the SM either reaches an end state (terminating the process) or all transitions require external input (e.g., waiting for further user input). Next, we provide a detailed introduction to each component.

B. Architecture

a) State machine: The SM in SHERPA systematically controls the LLM behavior by defining explicit states and transitions inspired by human best practices for the targeted task. Note that the state machine is a type of *data* in SHERPA, which supports dynamically altering the structure of the state machine without changing other components for an agent. Figure 2 shows the metamodel of the SM used in SHERPA.

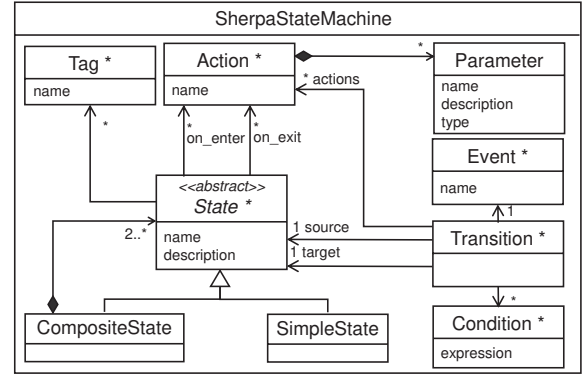


Fig. 2. The metamodel for state machines in SHERPA

The core components of the SM are its **states**. Although standard UML SMs include various types of states, such as parallel regions and history states, hierarchical states are particularly useful for decomposing complex tasks into simpler, manageable sub-tasks. Such decomposition aligns closely with the primary purpose of integrating human best practices into LLMs. Therefore, SHERPA employs two types of states: atomic **simple states** and hierarchical **composite states**. A composite state contains multiple sub-states. Each state includes a *name* and a textual *description*, which provide meaningful representations of the current state. These attributes are essential for the policy to evaluate and select potential transitions, especially when an LLM is used within the policy. Additionally, states may have optional **tags** denoting special behaviors. Currently, the supported tags include *start* and *end*, indicating the initial and final states, respectively.

The SM also incorporates a set of **transitions** defining its operational flow. Each transition explicitly connects a *source* state to a *target* state and specifies an **event** that triggers this transition. The triggering event may originate from a user message or be internally generated by the agent's policy. Transitions may also include optional guard conditions to provide finer-grained control over state transitions. Such guard conditions are represented using the **condition** class in the form of either a simple logical *expression* evaluated against the agent's belief or the *name* of an action with more sophisticated logic, such as invoking another LLM.

The behavior of the agent during state transitions is defined by **actions**. Typically, actions perform sub-tasks such as calling an LLM, retrieving relevant data from a database, or calling external tools for various types of specialized computation. Each action is characterized by a *name* and a set of **parameters**, which are defined by their *names*, *types*, and *descriptions*, including their expected data type and guidance for how they should be provided. Each parameter in an action can be one of the following two *types*, depending on how it should be provided: (1) *external parameters* are provided externally when the action is invoked, such as user input or provided by the policy through an LLM; (2) *internal parameters* are provided internally through the agent's belief during the execution of

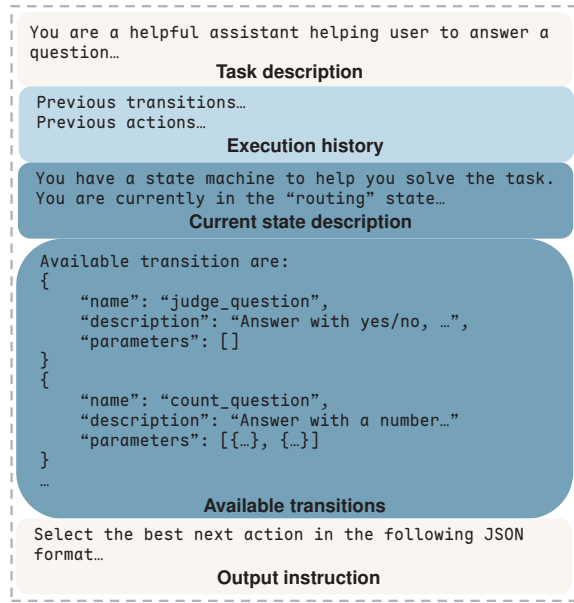


Fig. 3. Prompt for the LLM-based policy; the execution history including the trajectory and actions comes from the belief while the current state description and available transitions are provided by the SM.

the action. Actions can be integrated into the SM in multiple ways: they may be attached directly to transitions, triggering execution upon transition activation, or associated with states, executing when *entering* or *exiting* a particular state.

b) Policy: A *policy* in SHERPA determines the next transition for an agent by considering its current state and belief. The term policy is adapted from reinforcement learning literature, given its analogous role in decision-making [36]. Formally, let $s \in \mathcal{S}$ be the current state, $\mathcal{T}_s$ be the set of available transitions at state s , $\mathcal{E}$ be the set of possible events that can trigger transitions, and B be the agent's belief. A policy is defined as a mapping $\pi(s, \mathcal{T}_s, B) \rightarrow e$, where $e \in \mathcal{E}$ is the selected event along with any data required by the transitions.

Policies can be implemented using various approaches, including rule-based systems, a supervised machine learning model, reinforcement learning, or LLMs. In the current version of SHERPA, to demonstrate the concept, we introduce two primary policy types that can be combined flexibly to guide an agent's behavior. Notably, SHERPA can be easily extended to incorporate additional policies provided that they conform to the predefined policy interface.

Rule-based policy. A rule-based policy uses predefined mapping rules to determine the next transition based on the agent's current state and belief. These rules typically reference the current state and specific values stored within the belief of an agent. For example, a rule may specify that whenever a generated domain model includes an abstract class without subclasses, the policy should select a transition leading back to a state responsible for regenerating the domain model.

LLM-based policy. Inspired by the ReAct framework [18], the LLM-based policy utilizes an LLM to determine the next state transition. Figure 3 illustrates an example prompt used for

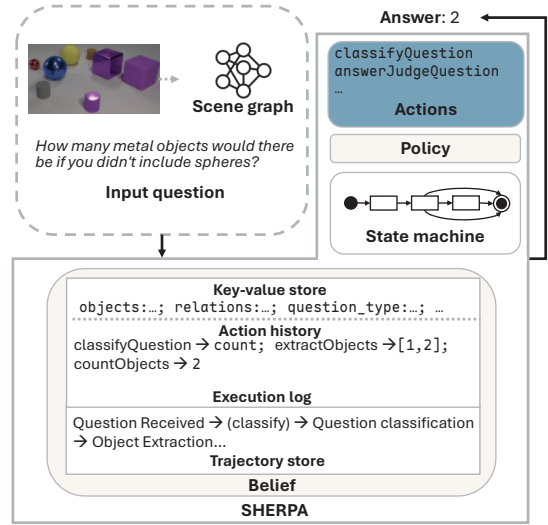


Fig. 4. Example of SHERPA for the scene graph-based question answering

selecting the next transition within the context of question answering. Upon policy initialization, a *task description* defining the overall task, along with an *output instruction* specifying the expected output, is provided. During execution, when the LLM-based policy is invoked, the relevant *execution history*, including the trajectory and previous actions, is extracted from the *belief*. To prevent exceeding the maximum token limit of LLMs, SHERPA uses a sliding-window mechanism to extract the most recent execution history that uses up to a fixed amount of tokens. Additionally, the SM provides the *current state*, including its description and available transitions. If any actions are associated with these transitions, or with exiting the current state or entering the subsequent state, the LLM is also prompted to generate the necessary *external input parameters* for these actions. Based on this information, the LLM is then prompted to select the most appropriate next transition.

Beyond these two policy types, SHERPA incorporates a *fast-forward* mechanism for efficiency. In situations where only one transition is available, due to either the SM's structure or the evaluation of guard conditions, the fast-forward mechanism automatically selects this transition, skipping potentially computationally intensive policy evaluations.

c) Belief: The *belief* of an agent in SHERPA is a structured data store to maintain information relevant to the current task. Compared to typical memory in LLM-based agents [29], which usually consists solely of a key-value store, the belief structure in SHERPA additionally includes a *trajectory store* and an *execution log*. The trajectory store maintains a sequence of state transitions that the agent has traversed, thereby providing essential context for decision-making. In parallel, the execution log records the history of actions performed by the agent, enabling tracking and retrospective analysis of behavior. The execution log also contains a key-value store preserving task-specific information that can be dynamically saved during action execution. Information in the belief is accessible during both transition selection by the policy and action execution.

C. Example

Figure 4 illustrates the integration of a routing SM (see Figure 5 (3.a)) for question answering from a scene graph. Specifically, the LLM is prompted with the question: “How many metal objects would there be if you didn’t include spheres?”. Note that the image is first converted into a textual scene graph, and the LLM’s task is to answer the given question based on this graph. Initial experiments indicate that LLMs frequently produce incorrect answers to counting questions. We hypothesize that this arises because the LLM simultaneously performs multiple complex subtasks: (1) interpreting the question, (2) extracting relevant objects from the scene graph, and (3) counting these extracted objects.

To address this challenge, we propose employing an SM to explicitly decompose the question-answering process into distinct subtasks based on the type of question. Specifically, relevant contextual information, such as the scene graph and question, is initially stored in the *belief*. Then, the LLM is prompted to determine the question type (*classifyQuestion* action within the *QuestionClassification* state). If identified as a counting question, the LLM is subsequently prompted to extract relevant objects from the scene graph (*extractObjects* action upon entering the *ObjectExtraction* state). After extraction, the number of objects is counted deterministically (*countObjects* action upon exiting the *ObjectExtraction* state). Throughout this procedure, SHERPA maintains a *trajectory store* to track state transitions, and an *execution log* that records the sequence of executed actions along with their corresponding inputs and outputs. Finally, the output of SHERPA is defined as the result of the final executed action.

IV. USE CASES

To demonstrate and assess the effectiveness of SHERPA, we examine three distinct tasks that can be automated using LLMs: *code generation*, *class name generation*, and *question-answering*. We specifically select these tasks due to their differences in data availability and human best practices. Code generation is widely recognized as a popular task for evaluating the capabilities of LLMs, supported by numerous available datasets [37]. Additionally, established human best practices, such as test-driven development, are already popular in software engineering, providing effective frameworks for addressing this task. In contrast, class name generation represents a typical modeling task. Although this task has well-established human best practices, it currently has significantly fewer available datasets. Finally, question-answering is a well-known task for LLM evaluation with many datasets [38]; however, unlike the previous two tasks, it lacks clearly defined human best practices for systematically solving this task generally.

A. Code generation

Task description. The code generation task involves generating code based on natural language descriptions. LLMs recently have emerged as promising tools for this task due to their strong code generation ability [37].

State machines. To systematically address the code generation problem using SHERPA, we implement two SMs (see Figure 5 (1)) based on how humans use tests to write code.

In the *test-driven* SM, we first ask the LLM to generate test cases based on the function description, after which the LLM is prompted to generate the solution function. A function is returned if it passes all the tests. If not, the process returns to the *Test Case Generated* state to retry. If no function passes all the tests after a certain budget is reached, the one that passes the most tests is returned.

The *agent coder* SM is based on the AgentCoder approach [39], with a single LLM performing both roles: programmer and test designer. The process begins with the LLM generating the function. Once the function is produced, the LLM then generates test cases. If the function does not pass all tests, the process retries from the *Start* state, generating a new candidate function and tests. If no function passes all the tests, the one that passes the most tests is returned.

The *agent coder* SM shows the applicability of SHERPA by replicating a previous approach. The *test-driven* SM can be seen as a refinement of the *agent coder* SM, where the repeated generation of test cases is avoided.

B. Class name generation

Task description. Domain model generation is an important MDE task where LLMs are increasingly popular [10]. This task involves generating a domain model based on a problem description. However, the evaluation of generated domain models is typically labor-intensive. Thus, we focus on generating class names in domain models to simplify this process.

State machines. To systematically address this generation problem using SHERPA, we implement two SMs, one of which is shown in Figure 5 (2). The main flow in the SM is motivated by the multi-step iterative generation approach (MIG) [40], which includes the following components for class names:

(1) *Classes and attributes generation.* In this composite state, the LLM is asked to generate all potential classes, including regular classes, abstract classes, and enumerations. The process begins by extracting relevant nouns from the problem description, which serve as candidate class names, attribute names, or enumeration literals. Subsequently, the approach categorizes these candidates into regular, abstract, and enumeration classes in a cascading manner. The final step involves assigning attributes to each class based on previous noun analysis and previously identified class types.

(2) *Pattern generation.* Patterns are an integral part of domain modeling. In this composite state, the approach performs *player-role* pattern identification and makes transitions based on the result using an LLM. If no designated pattern is detected, the SM transitions to the feedback generation state. Otherwise, it proceeds to generate the identified pattern and integrate the pattern into the model to align with modeling practice.

(3) *Generate feedback.* This composite state enhances the partial domain model through LLM self-reflection. The approach first generates feedback about the model, then iteratively improves the partial domain model based on this feedback.

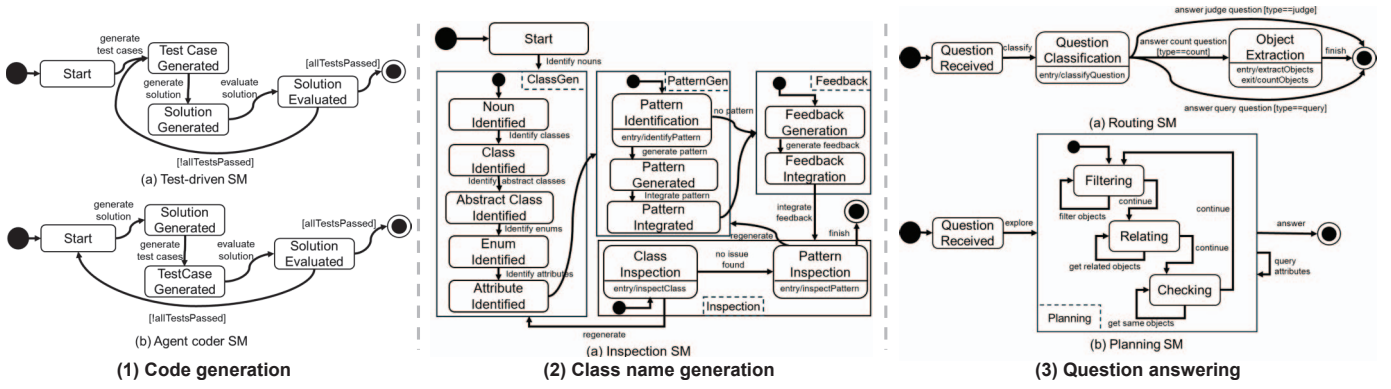


Fig. 5. SM design for the use cases; states for outputs and actions on the transitions are omitted for conciseness

Even though the focus is on class name generation, attributes and patterns are identified as they also help to identify classes.

The above processes are largely *linear* as there is no mechanism for the generation process to retry if some major mistakes are detected. To address this issue, we further introduce the *inspect model* composite state, providing transitions to return to the *identify classes and attributes* state or *identify pattern* state. The LLM is asked to examine the classes and determine if the process needs to go back to a previous state.

C. Question answering

Task description. Question answering is a classic natural language processing task. Due to LLMs' impressive natural language understanding capabilities, they are increasingly used for this task [38]. In this paper, we specifically focus on *scene graph-based* question answering, where the input context is provided as a scene graph. We categorize the questions into three types: (1) *judging* questions, which require determining whether a statement is true or false; (2) *counting* questions, which ask how many objects within the scene graph satisfy a particular condition; and (3) *querying* questions, which request specific attributes of objects within the scene graph. Due to the lack of established human best practices, we propose three SM approaches inspired by popular LLM-based methods [18], [41] (two of which are shown in Figure 5 (3)).

State machines. The *routing* SM follows the approach demonstrated in the example in Figure 4, representing a typical question-answering strategy using LLMs [41]. The core idea is to first classify the question type and then address it using methods tailored specifically for each type. For counting questions, the LLM is prompted to extract objects matching the question criteria, after which counting is performed deterministically. In contrast, for judging and querying questions, the LLM directly generates the answers.

We implement the widely adopted ReAct approach [18] for leveraging LLM reasoning with SHERPA as *ReAct* SM (not shown in the figure for conciseness). Here, the primary role of the LLM is to perform planning by sequentially calling predefined operations on the scene graph to gather relevant information. Thus, the SM consists of one state with self-transitions for these operations. The available operations are

defined as follows: (1) *filter*, to extract objects from the scene graph based on attribute values; (2) *relation*, to identify objects having specific relations to another object; (3) *checking*, to determine objects sharing a particular attribute with another object; and (4) *query*, to retrieve an attribute of a given object. Finally, there is another transition to the end state.

However, we hypothesize that providing too many action choices might overwhelm the LLM. Therefore, we propose the *planning* SM, a more structured version of the *ReAct* SM. This approach provides a predefined action sequence as heuristic based on observations from sampled questions. Specifically, the LLM first *filters* the scene graph based on attributes, then retrieves objects either *related* to those filtered objects or by *checking* objects sharing the same attributes. At any step, the LLM can *query* attributes of any object in the scene graph.

V. EVALUATION

Given a use case, there are multiple ways to implement best practices into an SM to guide the behavior of LLMs. Furthermore, while SHERPA can be used to implement existing LLM approaches, it is not trivial to determine the best design for a given use case. In this section, we evaluate popular approaches and their enhanced version implemented with SHERPA using the defined use cases. In particular, we target to address the following research questions:

- **RQ1** How does integrating state machines in SHERPA impact the performance of LLMs with different sizes compared with directly using LLMs?
- **RQ2** How does the state machine design and configuration affect the performance and cost of SHERPA?

A. Evaluation setup

a) LLMs: We evaluate SHERPA using four popular LLMs for all use cases as discussed in Section IV. Additionally, we choose one dedicated LLM for each use case that was designed to solve the task targeted by that specific use case.

Common LLMs. We evaluate our approach using two of the most recent LLMs from OpenAI: GPT-4o Mini [42] and GPT-4o [4]. These models demonstrate improved capability and performance on complex tasks compared to their predecessors,

while reducing inference costs. Nevertheless, their closed-source nature raises concerns regarding data privacy, security, and reproducibility. Thus, we also perform experiments with two state-of-the-art open-source models: Qwen2.5 7B [5], one of the most powerful LLMs deployable on consumer-grade GPUs, and Qwen2.5 72B, one of the most capable configurations within the Qwen LLM family [5]. Both LLMs exhibit better performance over similarly scaled open-source alternatives across a wide range of evaluation tasks.

Task-specific LLMs. We also select specialized LLMs tailored specifically to each use case. For the **code generation** task, we choose Qwen2.5 Coder (32B) [21], an open-source LLM designed for code generation, achieving top performance across multiple coding benchmarks. For the **class name generation** and **question answering** tasks, primarily challenging natural language-related capabilities, we evaluate Llama3.1 70B [22], another popular and powerful open-source LLM comparable to Qwen2.5 72b but outside the GPT and Qwen families.

b) Benchmarks: We evaluate the effectiveness of different approaches with the following benchmarks.

Code generation. To evaluate how SHERPA enhances the code generation process, we use the **HumanEval** benchmark [43]. The benchmark includes 164 programming problems and is widely adopted in LLM research to assess model performance in code generation [6]. In this benchmark, the LLM is provided with a natural language description and the header of a Python function and is tasked with generating the function body.

Class name generation. For class name generation, we use the automated domain modeling dataset proposed by Chen et al. [27] (**Modeling**). This dataset contains eight domain models with a total of 135 classes, spanning diverse domains and varying complexity levels. We specifically use classes from these domain models for evaluation.

Question answering. For the scene-based question answering task, we use human-curated questions from the **Clevr** dataset [44], [45]. Compared to template-generated questions, these human-curated questions offer greater linguistic diversity and complexity. We manually verify each question and exclude any questions that require visual context beyond the scene graph (e.g., object reflections). In total, we randomly sample 100 questions from the dataset, including 33 judgment-based, 33 counting-based, and 34 querying-based questions.

c) Metrics: We select metrics to evaluate the quality of results specifically for each use case.

Code generation. Each generated function is evaluated using the test suite from the HumanEval benchmark [43]. We use the *Pass@1* metric, measuring the proportion of problems solved by passing the test suite within 1 attempt.

Class name generation. Evaluating class names can be challenging due to variations in synonyms and naming conventions. Relying solely on exact matches typically significantly underestimates the true generation quality. To overcome this limitation, we adopt an embedding-based evaluation method [46], which has been shown to correlate well with human judgments. This approach measures the cosine similarity between generated and

reference labels to create overall scores (precision (P), recall (R), and F_1 -score (F_1)) for the set of generated classes.

Question answering. For Clevr, we measure answer accuracy (*ACC*). As the answers belong to a finite set of categories (e.g., numbers, attribute values, etc.), we use *exact match* accuracy between the generated and ground truth answers.

Furthermore, the iterative nature of the SMs can potentially lead to increased cost for LLM invocations, especially when using an LLM API or when the LLM is deployed on a cloud service. To evaluate this aspect, we also measure the *average number of LLM calls* required for each use case.

d) Implementation details: We set the number of maximum state transitions to 10 for each use case. To overcome the variability of the LLM's output, we choose a 0.01 generation temperature and measure the average performance over 3 runs. The repository for the evaluation is available online [47].

B. RQ1: Comparison with direct methods

a) Rationale and setup: When considering building domain-specific SMs to control LLM behavior for a particular task, it is crucial to understand the situations under which such an approach can be beneficial. This research question addresses this aspect by comparing the performance of SMs powered by SHERPA against the *Direct* approach, which relies solely on prompting techniques. We investigate two sub-questions:

- **RQ1.1:** How do SMs in SHERPA influence the quality of outputs from LLMs compared to *Direct*?
- **RQ1.2:** How does the impact of SMs vary across different LLM sizes within the same LLM family?

For each use case, we select appropriate prompting techniques for the *Direct* method. Specifically, for the code generation task, we choose zero-shot prompting using a widely adopted prompt that has demonstrated effectiveness across various GPT models [48]. For the class name generation task, we utilize one-shot prompting, providing a single illustrative example from a domain not included in the testing set, demonstrating the nature of the task and the expected format of the output. Lastly, for the question answering task, we adopt chain-of-thought prompting [16], as answering questions based on scene graphs typically requires step-by-step reasoning.

For the SM in SHERPA, we use the first SM illustrated in Figure 5 for each use case (specifically, the test-driven, inspection, and routing SMs). Note that different SM designs can significantly influence the quality of the final output (see RQ2). For RQ1, our goal is to understand the general impact of using an SM. Therefore, we selected SMs representing straightforward designs that can be easily constructed and implemented by software engineers.

b) Impact of SM integration: Table I presents the performance comparison between SHERPA and *Direct* across all use cases and LLMs. For each use case-LLM pair, we underline the better-performing method and highlight the best overall performance for each use case in bold. In general, empowering LLMs with SMs in SHERPA improves performance compared to the *Direct* approach in 12 out of 15 cases, while being comparable in the remaining three. SHERPA also achieves the

TABLE I
PERFORMANCE COMPARISON BETWEEN DIRECT AND SM-BASED METHODS.

	HumanEval		Modeling						Clevr	
	Direct	SHERPA	Direct			SHERPA			Direct	SHERPA
	<i>Pass@1</i>	<i>Pass@1</i>	<i>P</i>	<i>R</i>	<i>F₁</i>	<i>P</i>	<i>R</i>	<i>F₁</i>	<i>ACC</i>	<i>ACC</i>
GPT-4o Mini	85.37	88.82	88.89	63.20	73.36	85.22	70.77	76.77	86.00	86.67
GPT-4o	91.26	90.24	92.98	55.24	68.90	92.82	58.65	71.16	90.67	88.33
Qwen2.5 7B	83.13	85.37	81.15	47.52	57.38	89.75	62.06	71.57	70.00	75.33
Qwen2.5 72B	84.76	90.65	90.41	56.78	68.98	89.35	73.71	80.07	86.67	86.67
Task-Specific LLM	89.84	91.26	88.76	64.07	73.85	88.51	67.68	75.80	83.00	84.67

highest performance in two out of three use cases, slightly underperforming GPT-4o in the Clevr dataset.

Analyzing the impact for each task individually, SHERPA outperforms Direct in code generation for four out of five LLMs, showing an average improvement of 3.25 percentage points (i.e., $SHERPA - Direct = 3.25$). The overall improvement in LLMs other than GPT-4o is primarily due to the explicit test-generation step included in the SM, which validates generated code against these tests.

For class name generation, integrating the SM notably improves recall but occasionally leads to decreases in precision. The iterative refinement in the SM results in more class names being discovered, but also occasionally introduces irrelevant class names. Nonetheless, SHERPA consistently achieves higher overall F_1 for all LLMs, with an average improvement of 6.58 percentage points. This substantial gain can be attributed to the SM’s alignment with existing human best practices.

In the question-answering task, SHERPA achieves higher accuracy in three out of five LLMs, with an average improvement of 2.56 percentage points. Interestingly, it does not bring benefit with the two largest LLMs, likely because these models already effectively manage CLEVR’s question-answering tasks for different question types. Smaller LLMs, however, clearly benefit from the routing logic provided by the SM, which helps them address different aspects of each question type.

RQ1.1. Integrating SMs within SHERPA generally enhances LLM performance compared to the Direct method, improving the performance in 12 out of 15 cases. The most significant benefit occurs in the class name generation task, with an average improvement of 6.58 percentage points, which can be attributed to the fact that there exists an established best practice for this task.

c) *Impact of LLM size:* Examining the impact of LLM size on performance, we find that integrating SMs provides more consistent benefits for smaller LLMs. Within both the Qwen and GPT families, SHERPA consistently outperforms Direct for smaller LLMs, while it improves performance in only three out of six cases for larger variants. We hypothesize that larger LLMs possess sufficient capabilities to handle the tasks effectively without integrating SMs. Indeed, larger LLMs consistently outperform their smaller counterparts across all tasks, except in one of the class name generation cases, where GPT-4o Mini unexpectedly outperforms GPT-4o. We suspect

that the benefits observed in smaller LLMs may also occur in larger LLMs when applied to more complex tasks.

During early experiments, we iteratively developed the SM with smaller LLMs to reduce inference costs. As a result, the final SM design may favor smaller LLMs and limit effectiveness with larger variants, suggesting that SMs should be tailored to the target LLM to maximize their benefits.

RQ1.2. The impact of SMs on LLM performance is more pronounced in smaller models. Specifically, SHERPA consistently outperforms Direct for smaller LLMs, while it may damage the performance for larger LLMs. This observation indicates that the SM design should be tailored to the specific LLM to maximize effectiveness.

C. RQ2: Impact of SM design

a) *Rationale and setup:* RQ1 demonstrated that integrating SMs with SHERPA generally improves the performance. However, the SM itself may also influence the effectiveness. Moreover, the multi-step nature of SMs increases the number of LLM calls and thus impacts the overall cost of LLM executions. In SHERPA, SM design is decoupled from action implementation, enabling updating the SM without modifying the underlying implementation to reach cost targets. In this RQ, we analyze how different SMs affect both performance and cost. Specifically, we address the following sub-questions:

- **RQ2.1:** How do different SM configurations in SHERPA influence the quality of LLM-generated outputs?
- **RQ2.2:** How do different SM configurations in SHERPA impact the number of LLM invocations?

We use SMs for the use cases described in Section IV to answer these questions. For the code generation task, we compare the *test-driven* SM, which attempts to reduce LLM calls by avoiding repeatedly generating new test cases when a generated function fails, to the *agent coder* SM. In the class name generation task, we compare the *inspection* SM, which introduces an additional *inspect model* composite state, to the *MIG* approach. In the question answering task, we examine the *routing* SM and *ReAct* SM. We also evaluate the *planning* SM that further enforces a predefined operation sequence commonly used in this task to avoid wasting LLM invocations.

b) *Impact on performance:* Figure 6 presents the performance of various SM configurations across different use cases. Overall, all SM-based approaches generally outperform Direct, although the specific performance varies. For the

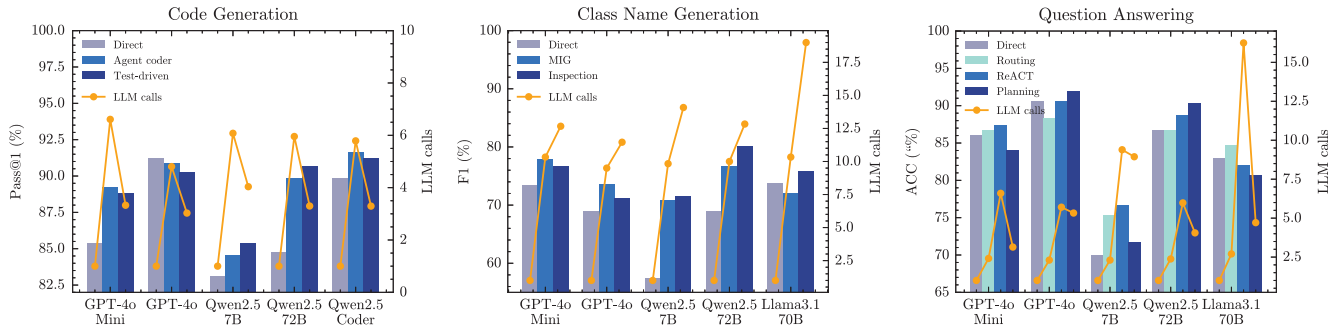


Fig. 6. Comparison of different SM designs and the *Direct* method with respect to performance and number of LLM calls

code generation task, the *test-driven* SM achieves comparable performance to the *agent coder* SM, even surpassing it with Qwen2.5-7B and Qwen2.5 72B, indicating that repeatedly generating test cases may be unnecessary for the code generation scenario, particularly with Qwen models.

In the class name generation task, introducing the additional inspection state in the *Inspection* SM enhances performance over the *MIG* SM for both Qwen and Llama LLMs; however, it slightly degrades the performance for GPT-4o variants. Nevertheless, the combination of the *Inspection* SM with Qwen2.5-72B yields the highest overall performance.

For the question answering task, we examine two distinct SM modeling strategies. While the *routing* SM outperforms the *Direct* approach in three out of five cases, separating planning and information extraction in the *ReAct* SM results in further performance gains in four out of five LLMs. Additionally, enforcing a predefined sequence of operations in the *planning* SM enhances performance specifically for the two strongest LLMs tested in this task: GPT-4o and Qwen2.5 72B. We hypothesize that less capable LLMs might lack sufficient usage of predefined operation order (by consistently firing the *continue* transitions), making the *planning* SM less helpful.

RQ2.1. The SMs notably impact SHERPA performance, varying depending on the specific LLM size and LLM architecture. For tasks with well-defined best practices (e.g., code generation, class name generation), the impact seems to vary for different LLM families, while for tasks with less defined best practices (e.g., question answering), the impact seems to depend more on LLM size.

c) *Impact on cost:* The orange lines in Figure 6 indicate the cost in terms of the average number of LLM calls required by each SM design. Across all tested LLMs and tasks, a clear trend emerges: SMs designed explicitly to minimize LLM calls indeed result in a reduced number of invocations. Specifically, the *test-driven* SM achieves approximately 50% fewer LLM calls on average compared to the *agent coder* SM. Similarly, the pre-defined action orders in the *planning* SM also reduce the number of LLM calls compared to the *ReAct* SM, with a particularly significant reduction observed for Llama3.1 70B. Conversely, the additional *inspect model* state in the *Inspection*

SM increases the number of LLM calls relative to the *MIG* SM, since this state explicitly prompts the LLM to inspect outputs and potentially revert to previous states.

RQ2.2. The configuration of SMs can influence the number of LLM calls in SHERPA. Generally, configurations explicitly designed to reduce LLM calls achieve notable reductions in cost.

D. Discussion

Applicability. In RQ1, we demonstrate that integrating SMs into SHERPA consistently enhances LLM performance across three tasks, with the most pronounced improvements for code generation and class name generation. These tasks notably have clear human best practices, which effectively guide the LLM by systematically decomposing the tasks. In contrast, the question-answering task is more general and lacks well-defined best practices, resulting in relatively modest performance gains. We suspect that integrating SMs using SHERPA is more effective for tasks with established best practices, especially for use cases lacking enough training data (e.g., class name generation).

Impact of state machine configuration. In RQ2, we observe that the SM design significantly influences LLM performance, although this influence varies according to the LLM family and size. Due to this experimental nature, the separation of SM design from action implementation in SHERPA is especially valuable, enabling rapid experiments and optimization of different SM designs without altering underlying implementations.

Furthermore, SMs designed to optimize the number of LLM calls consistently achieve reductions across all tested tasks, indicating more predictable costs when using SHERPA. Importantly, these optimized SMs maintain performance comparable to unoptimized SMs, showing that cost savings do not come at the expense of effectiveness. This balance is particularly beneficial when using strong LLMs from an API, where computational costs can be substantial. We believe the explicit structure of SMs enables engineers to more accurately predict and manage LLM execution costs, potentially integrating them effectively with traditional model-checking techniques on SMs [33].

E. Threats to validity

Internal validity. LLM outputs may be non-deterministic depending on the configuration and hardware. To mitigate

this issue, we use a low sampling temperature (0.01) for all experiments and average results across three independent runs. Additionally, the performance of LLMs can vary based on prompting techniques. To minimize this variation, we consistently followed established techniques, such as chain-of-thought prompting [16] and few-shot prompting [3] for different approaches compared in the experiments.

External validity. The impact of SMs could vary across different domains and tasks. In this paper, we evaluate SMs on three distinct use cases. However, the observed impacts may differ for other domains. We leave the exploration of SMs' impacts on other tasks and domains as future work.

Construct validity. The evaluation metrics adopted in this paper may raise a threat. For the *Pass@1* metric, passing all tests does not necessarily guarantee the correctness of the generated functions. Full correctness evaluation may require complex analysis of generated code. Nevertheless, this metric is widely recognized and used in existing research [49]. For the class name generation and question-answering tasks, we likewise selected the most commonly used metrics [27], [44].

VI. RELATED WORK

MDE for LLMs. Applying MDE principles to enhance LLM applications is an emerging research area. Clarisó et al. [50] introduce *Impromptu*, a domain-specific language (DSL) designed for defining prompts. Several frameworks [51], [52] also use DSL to define conversational agents with LLMs. For testing LLMs, Morales et al. [53] present *LangBite*, a model-driven approach for specifying ethical requirements and automating the testing of ethical biases in LLMs.

SHERPA also uses a model-driven approach to optimize the use of LLMs. While methods like *Impromptu* primarily focus on the specific prompt, SHERPA works at a higher level of abstraction that constrains the flow of LLM-based applications.

LLMs for MDE. The use of LLMs for automating MDE tasks has emerged as a prominent research area [10]. Notably, these models have been effectively used for model completion or recommendation [54], model query generation [49], [55], domain model generation [27], [56], among others [10].

Most approaches using LLMs for automating MDE tasks resemble the *direct* approach, typically involving a single prompt or a linear sequence of prompts. For example, Chen et al. [27] evaluate various LLMs for domain model generation using few-shot prompting. Similarly, Abukhalaf et al. [49] use different prompting strategies to generate OCL queries.

The use of LLMs in these approaches are more similar to the *Direct* baselines used in this paper (for which SM integration shows benefit in the majority of the cases), we believe that, with well-designed SMs tailored for each scenario, SHERPA could be used to further enhance these MDE tasks.

Structured workflow for LLMs. Many approaches [12], [18], [28], [57] have been proposed to improve the performance of LLMs by using a structured workflow. The Reflexion framework [57] enhances LLM agents through self-feedback. In the ReAct framework [18], LLMs generate reasoning traces and task-specific actions. The retrieval-augmented generation

(RAG) using various tools has also been explored through various approaches [12]–[15]. To better manage external tools and enhance modularity, researchers have explored multi-agent systems [58]–[61]. Specifically, Wu et al. [58] introduce AutoGen, which uses multiple customizable and interactive LLM agents to collaboratively accomplish tasks.

SHERPA can also be seen as a way to integrate structured workflows with LLMs. Unlike previous methods, where workflows are either fixed to specific use cases or tightly coupled with the implementation, SHERPA proposes a general approach to define workflows and can be used to implement existing approaches such as RAG and ReACT.

State machines in LLMs. Relatively few research efforts have focused on integrating LLMs with state machine-like workflows [19], [20]. Wu et al. [19] propose StateFlow, a novel LLM-based task-solving framework that models complex task-solving processes as state machines, where transitions between states are governed by heuristic rules or LLM-driven decisions, with actions performed within each state. Another notable effort is the State Machine of Thoughts [20], which uses a state machine to track experiences from previous reasoning trajectories. Langchain [28] and similar frameworks [58], [62]–[64] build agentic applications with LLMs using SM-like workflows or pipelines. However, these frameworks typically treat the workflow as pre-compiled and static, making it difficult to modify the workflow dynamically at runtime.

Compared to existing approaches, SHERPA introduces several improvements: (1) it uses a hierarchical SM to support more modular structure definition; (2) it decouples SM design from action implementation, enabling better modularity; (3) it supports various policies for navigating the SM, including both LLM and rule-based approaches; and (4) it treats SMs as data, allowing them to be dynamically updated.

VII. CONCLUSION

In this paper, we propose SHERPA, a model-driven framework designed to enhance the performance of LLMs on complex tasks using domain-specific best practices with hierarchical SMs. By structuring execution through SMs, SHERPA provides more flexible and fine-grained control over the behavior of LLM-based applications. We demonstrate the applicability and effectiveness of SHERPA across diverse tasks. The results show that, although LLM performance is generally improved by SMs, it is highly influenced by the SM design. While the SM designs impact task performance differently depending on the LLM, specially designed SMs reduce the execution cost while maintaining a similar performance. Such result makes the separation of SM definition and action implementation in SHERPA particularly useful to support rapid experiments.

In future work, we plan to integrate different textual languages for SMs to simplify the design process. We also intend to explore applications of SHERPA in different domains beyond the three use cases. Additionally, investigating how SHERPA can be combined with complementary techniques, such as reinforcement learning-based policies, may further improve its performance and broaden its applicability.

REFERENCES

- [1] D. Harel, "Statecharts: a visual formalism for complex systems," *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987.
- [2] I. Jacobson and J. R. G. Booch, "The Unified Modeling Language reference manual," *Pearson*, 2021.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [4] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "GPT-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [5] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
- [6] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *arXiv preprint arXiv:2406.00515*, 2024.
- [7] P. Shailendra, R. C. Ghosh, R. Kumar, and N. Sharma, "Survey of large language models for answering questions across various fields," in *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)*, vol. 1. IEEE, 2024, pp. 520–527.
- [8] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of LLM agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024.
- [9] M. Aghzal, E. Plaku, G. J. Stein, and Z. Yao, "A survey on large language models for automated planning," *arXiv preprint arXiv:2502.12435*, 2025.
- [10] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubel, "On the use of large language models in model-driven engineering," *Software and Systems Modeling*, pp. 1–26, 2025.
- [11] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [13] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023.
- [14] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 7969–7992.
- [15] O. Khattab, K. Santhanam, X. L. Li, D. Hall, P. Liang, C. Potts, and M. Zaharia, "Demonstrate-Search-Predict: Composing retrieval and language models for knowledge-intensive nlp," *arXiv preprint arXiv:2212.14024*, 2022.
- [16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, brian ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain of thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022.
- [17] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, 2023.
- [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.
- [19] Y. Wu, T. Yue, S. Zhang, C. Wang, and Q. Wu, "StateFlow: Enhancing LLM task-solving through state-driven workflows," in *First Conference on Language Modeling*, 2024.
- [20] J. Liu and J. Shuai, "SMoT: Think in state machine," *arXiv preprint arXiv:2312.17445*, 2023.
- [21] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2. 5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.
- [22] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [24] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu *et al.*, "Instruction tuning for large language models: A survey," *arXiv preprint arXiv:2308.10792*, 2023.
- [25] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [26] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations*, 2024.
- [27] K. Chen, Y. Yang, B. Chen, J. A. H. López, G. Mussbacher, and D. Varró, "Automated domain modeling with large language models: A comparative study," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 162–172.
- [28] "LangGraph: Building knowledge graphs with language models," <https://langchain-ai.github.io/langgraph/>, accessed: 2024-09-30.
- [29] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.
- [30] G. H. Mealy, "A method for synthesizing sequential circuits," *The Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, 1955.
- [31] E. Domi, B. Pérez, Á. L. Rubio *et al.*, "A systematic review of code generation proposals from state machine specifications," *Information and Software Technology*, vol. 54, no. 10, pp. 1045–1066, 2012.
- [32] A. R. Van Cam Pham, S. Gérard, and S. Li, "Complete code generation from UML state machine," in *Proceedings of the 5th International Conference on Model-Driven Engineering and Software Development*, vol. 1, 2017, pp. 208–219.
- [33] R. Alur and M. Yannakakis, "Model checking of hierarchical state machines," *ACM SIGSOFT Software Engineering Notes*, vol. 23, no. 6, pp. 175–188, 1998.
- [34] "PlantUML at a glance," <https://plantuml.com/>.
- [35] T. C. Lethbridge, A. Forward, O. Badreddin, D. Brestovansky, M. Garzon, H. Aljamaan, S. Eid, A. H. Orabi, M. H. Orabi, V. Abdelzad *et al.*, "Umple: Model-driven development for open source and education," *Science of Computer Programming*, vol. 208, p. 102665, 2021.
- [36] R. S. Sutton, A. G. Barto *et al.*, *Reinforcement learning: An Introduction*. MIT press Cambridge, 1998.
- [37] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [38] M. Yue, "A survey of large language model agents for question answering," *arXiv preprint arXiv:2503.19213*, 2025.
- [39] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2023.
- [40] Y. Yang, B. Chen, K. Chen, G. Mussbacher, and D. Varró, "Multi-step iterative automated domain modeling with large language models," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 587–595.
- [41] C. Varogot-Reille, C. Bouvard, A. Gourru, M. Ciancone, M. Schaeffer, and F. Jacquenet, "Doing more with less—implementing routing strategies in large language model-based systems: An extended survey," *arXiv preprint arXiv:2502.00409*, 2025.
- [42] OpenAI, "GPT-4o mini: advancing cost-efficient intelligence," <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>.
- [43] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [44] J. Johnson, B. Hariharan, L. Van Der Maaten, L. Fei-Fei, C. Lawrence Zitnick, and R. Girshick, "CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 2901–2910.

- [45] J. Johnson, B. Hariharan, L. Van Der Maaten, J. Hoffman, L. Fei-Fei, C. Lawrence Zitnick, and R. Girshick, "Inferring and executing programs for visual reasoning," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2989–2998.
- [46] K. Chen, B. Chen, Y. Yang, G. Mussbacher, and D. Varró, "Embedding-based automated assessment of domain models," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 87–94.
- [47] B. Chen, K. Chen, J. A. H. López, G. Mussbacher, D. Varró, and A. Feizpour, "Artifact-sherpa4modeling." [Online]. Available: <https://zenodo.org/records/16338136>
- [48] "Performance of ChatGPT on HumanEval," <https://github.com/saschschramm/chatgpt?tab=readme-ov-file#performance>, accessed: 2025-03-23.
- [49] S. Abukhalaf, M. Hamdaqa, and F. Khomh, "On Codex prompt engineering for OCL generation: an empirical study," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023, pp. 148–157.
- [50] R. Clarisó and J. Cabot, "Model-driven prompt engineering," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 47–54.
- [51] J. S. Cuadrado, S. Pérez-Soler, E. Guerra, and J. de Lara, "Automating the development of task-oriented llm-based chatbots," in *ACM Conversational User Interfaces 2024, CUI 2024, Luxembourg, July 8-10, 2024*. ACM, 2024, p. 11.
- [52] J. Zhang and I. Arawjo, "Chainbuddy: An ai-assisted agent system for generating LLM pipelines," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, CHI 2025, YokohamaJapan, 26 April 2025- 1 May 2025*. ACM, 2025, pp. 241:1–241:21.
- [53] S. Morales, R. Clarisó, and J. Cabot, "A DSL for testing LLMs for fairness and bias," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 203–213.
- [54] M. B. Chaaben, L. Burgueño, and H. Sahraoui, "Towards using few-shot prompt learning for automating model completion," in *2023 IEEE/ACM 45th international conference on software engineering: New ideas and emerging results (ICSE-NIER)*. IEEE, 2023, pp. 7–12.
- [55] J. A. H. López, M. Földiák, and D. Varró, "Text2VQL: teaching a model query language to open-source language models with ChatGPT," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, 2024, pp. 13–24.
- [56] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML," *Software and Systems Modeling*, vol. 22, no. 3, pp. 781–793, 2023.
- [57] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, 2024.
- [58] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM applications via multi-agent conversation framework," in *First Conference on Language Modeling*, 2024.
- [59] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024.
- [60] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, Z. Tu, and S. Shi, "Encouraging divergent thinking in large language models through multi-agent debate," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 17 889–17 904.
- [61] L. Wang and H. Zhong, "LLM-SAP: Large language models situational awareness-based planning," in *2024 IEEE International Conference on Multimedia and Expo Workshops (ICMEW)*, 2024, pp. 1–6.
- [62] CrewAI Inc., "CrewAI: Fast and flexible multi-agent automation framework," <https://github.com/crewAIInc/crewAI>, 2025, accessed: 2025-07-24.
- [63] n8n Contributors, "n8n: Secure workflow automation for technical teams," <https://github.com/n8n-io/n8n>, 2025, accessed: 2025-07-24.
- [64] deepset-ai, "Haystack: The production-ready open secure ai framework," [urlhttps://github.com/deepset-ai/haystack](https://github.com/deepset-ai/haystack), 2025, accessed: 2025-07-24.